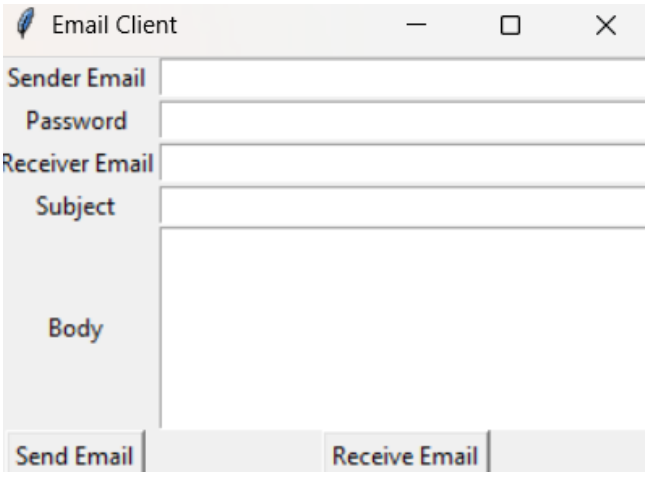
Final Project
Email Client Application

Part 1:

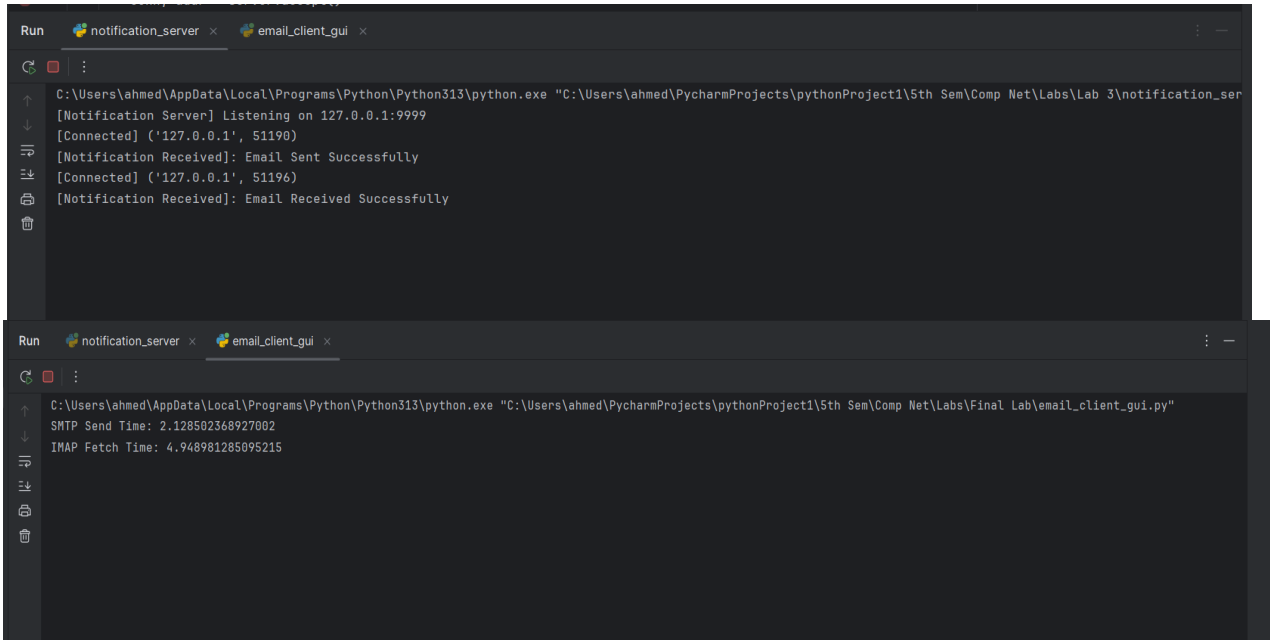
In this part, we implemented a Python-based email client application that can send and receive emails using standard email protocols. The application establishes a TCP connection with a mail server, communicates using the SMTP protocol to send emails and the IMAP protocol to receive emails, and properly closes the TCP connection after completing each operation.





Part 2:

In this part, a simple TCP notification server was implemented.
The server listens on port 9999 and waits for incoming TCP connections from the email client.
After sending or receiving an email, the email client establishes a TCP connection with the notification server and sends a short status message such as “Email Sent Successfully” or “Email Sending Failed”.
The server receives the message, prints it, and closes the connection properly.

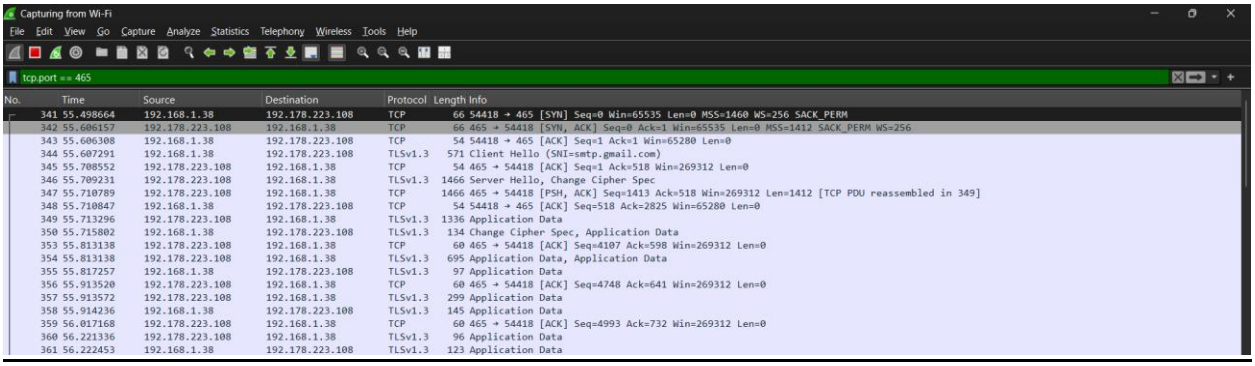


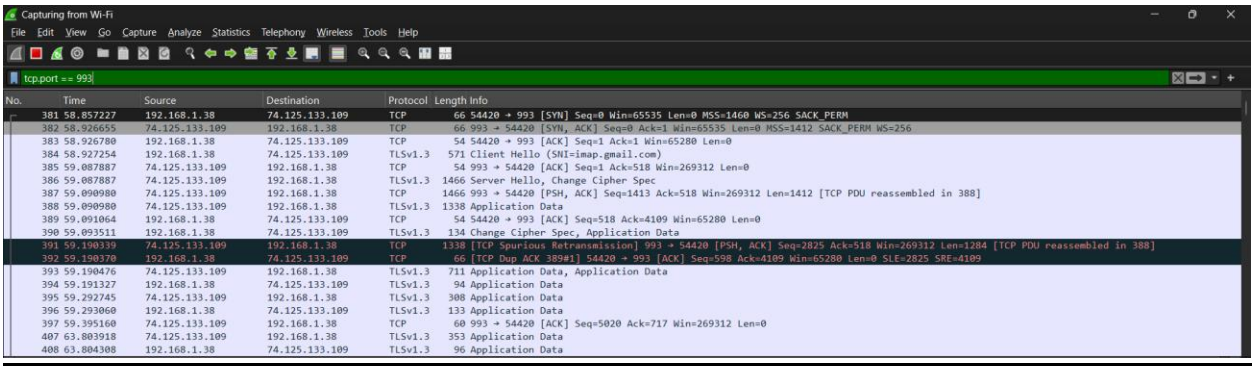
Part 3:

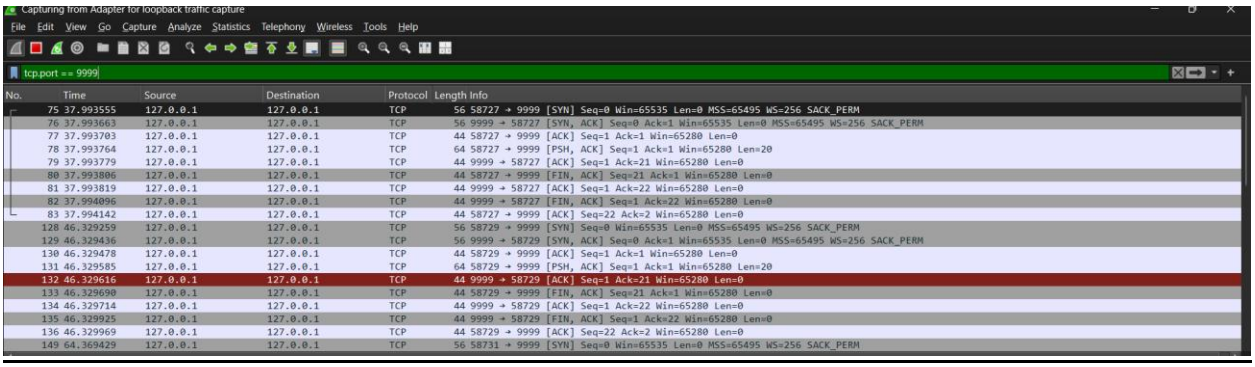
Wireshark was used to capture and analyze the network packets generated during the execution of the email client application.

Packet capturing was started before running the application, and specific filters were applied to observe SMTP, IMAP, and TCP notification traffic.

The captured packets show the TCP connection establishment, protocol commands, encrypted payloads, and proper connection termination.







Part 4:

1. Latency

Latency was measured as the total time taken to complete each operation. The time measurement was calculated using Python's time.time() function by recording the start and end times of each operation.

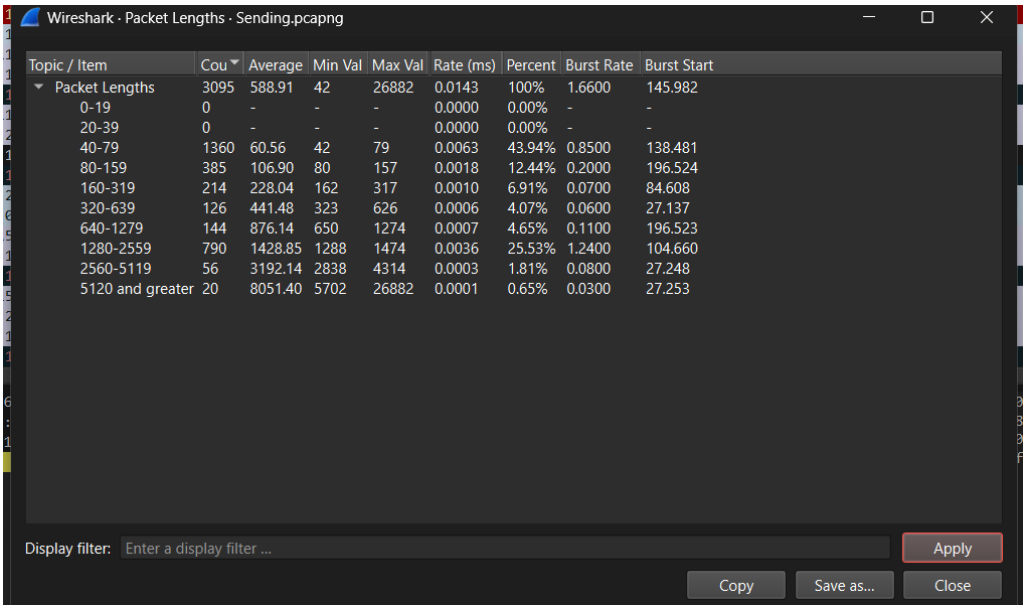
2.Packet Count

The total number of packets sent and received was obtained from Wireshark by counting the packets matching each protocol filter.

3. Throughput

Throughput was calculated using the following formula:

Throughput = Total Bytes Transmitted / Total Time Taken



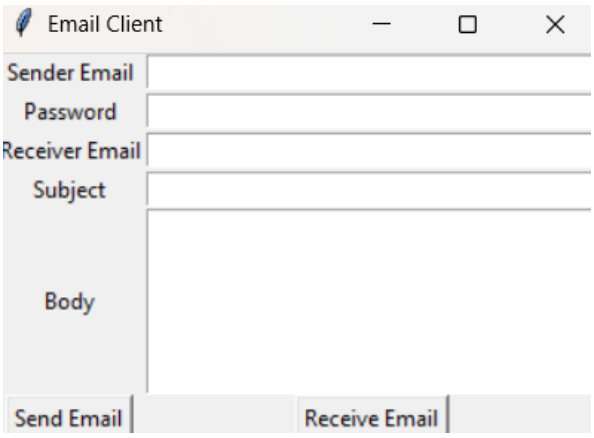
Part 5:

A graphical user interface (GUI) was implemented using Python's Tkinter library. The GUI allows the user to enter email credentials, recipient address, subject, and body, and provides buttons for sending and receiving emails.

Additionally, push notifications were implemented using the Plyer library. When an email is sent or received successfully, a system notification is displayed to alert the user.

notification_server.py email_client_gui.py

```
112
113
114  # ================= GUI ===================
115  root = Tk()
116  root.title("Email Client")
117
118  Label(root, text="Sender Email").grid(row=0, column=0)
119  Label(root, text="Password").grid(row=1, column=0)
120  Label(root, text="Receiver Email").grid(row=2, column=0)
121  Label(root, text="Subject").grid(row=3, column=0)
122  Label(root, text="Body").grid(row=4, column=0)
123
124  sender_entry = Entry(root, width=40)
125  password_entry = Entry(root, show="*", width=40)
126  receiver_entry = Entry(root, width=40)
127  subject_entry = Entry(root, width=40)
128  body_text = Text(root, width=30, height=6)
129
130  sender_entry.grid(row=0, column=1)
131  password_entry.grid(row=1, column=1)
132  receiver_entry.grid(row=2, column=1)
133  subject_entry.grid(row=3, column=1)
134  body_text.grid(row=4, column=1)
135
136  Button(root, text="Send Email", command=send_email).grid(row=5, column=0)
137  Button(root, text="Receive Email", command=receive_email).grid(row=5, column=1)
138
139  root.mainloop()
```



Code:

1.

notification_server.py × email_client_gui.py

```python

import socket

HOST = '127.0.0.1'
PORT = 9999

def start_notification_server():  1 usage
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(1)

    print(f"[Notification Server] Listening on {HOST}:{PORT}")

    while True:
        conn, addr = server.accept()
        print(f"[Connected] {addr}")

        message = conn.recv(1024).decode()
        print(f"[Notification Received]: {message}")

        conn.close()

if __name__ == "__main__":
    start_notification_server()

```

2.

notification_server.py email_client_gui.py ×

```python
import smtplib
import imaplib
import email
import socket
import time
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from tkinter import *
from plyer import notification

SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 465

IMAP_SERVER = "imap.gmail.com"
IMAP_PORT = 993

NOTIFY_HOST = "127.0.0.1"
NOTIFY_PORT = 9999


def send_notification(msg):  3 usages
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((NOTIFY_HOST, NOTIFY_PORT))
        s.send(msg.encode())
        s.close()
    except:
        pass

```

notification_server.py email_client_gui.py ×

```python
def send_email():  1 usage
    start_time = time.time()

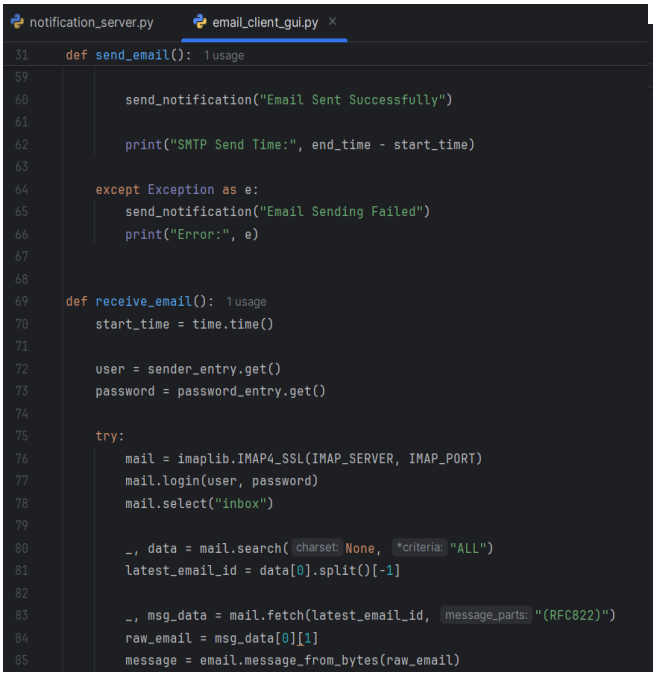
    sender = sender_entry.get()
    password = password_entry.get()
    receiver = receiver_entry.get()
    subject = subject_entry.get()
    body = body_text.get( index1: "1.0", END)

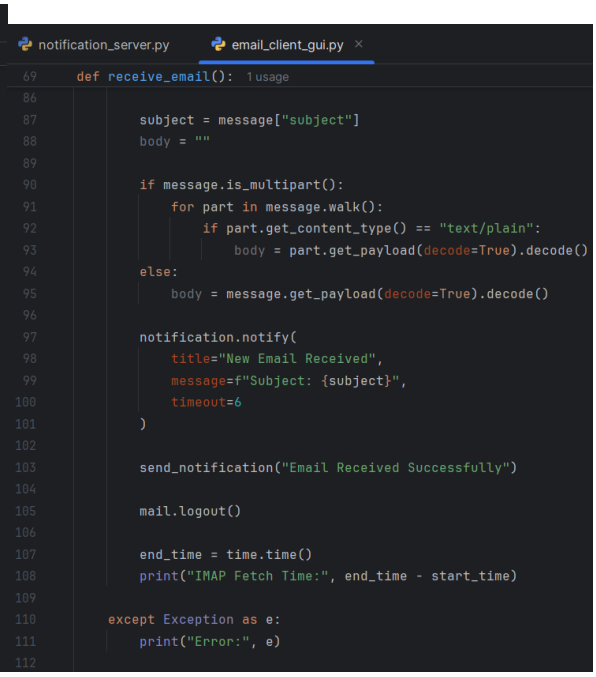
    try:
        msg = MIMEMultipart()
        msg["From"] = sender
        msg["To"] = receiver
        msg["Subject"] = subject
        msg.attach(MIMEText(body,  _subtype: "plain"))

        server = smtplib.SMTP_SSL(SMTP_SERVER, SMTP_PORT)
        server.login(sender, password)
        server.send_message(msg)
        server.quit()

        end_time = time.time()

        notification.notify(
            title="Email Client",
            message="Email Sent Successfully",
            timeout=5
        )
```





By:

Ahmed ElSawy	2305238
Moamen Amer	2305520
Moataz Amer	2305219
Mohab Ahmed	2305386